CS0054
(INTELLIGENT SYSTEMS)

EXERCISE

4

NATURAL LANGUAGE PROCESSING (NLP)

Name:	Eric V. Tividad Jr.
Section:	AN 42
Professor:	Ms. Crisola Tan

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design and evaluate solutions for computing problems. [PO: E]

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- To design Intelligent system based on methods learned. [CLO: 3]

III. INTENDED LEARNING OUTCOME/S OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Know and understand natural language processing & language technology.
- Familiarize with the approaches to NLP.
- Learn sample applications of NLP.

IV. BACKGROUND INFORMATION

What is NLP?

Working in Data Science and having a background in Technical Writing, I was drawn to the field of Natural Language Processing (NLP). Machines understanding language fascinates me, and I often ponder which algorithms Aristotle would have used to build a rhetorical analysis machine if he had the chance. If you're new to Data Science, getting into NLP can seem complicated, especially since there have been so many recent advancements in the field. It is hard to know where to start.

What can Machines Understand?

While a computer can be actually quite good at finding patterns and summarizing documents, it must transform words into numbers before making sense of them. This transformation is needed because machines "learn" thanks to mathematics, and math doesn't work very well on words. Before transforming the words into numbers, they are often cleaned of things like special characters and punctuation, and modified into forms that make them more uniform and interpretable.

What are Word Clouds?

Word clouds (also known as text clouds or tag clouds) work in a simple way: the more a specific word appears in a source of textual data (such as a speech, blog post, or database), the bigger and bolder it appears in the word cloud.

A word cloud is a collection, or cluster, of words depicted in different sizes. The bigger and bolder the word appears, the more often it's mentioned within a given text and the more important it is.

Also known as tag clouds or text clouds, these are ideal ways to pull out the most pertinent parts of textual data, from blog posts to databases. They can also help business users compare two different pieces of text to find the wording similarities between the two.

For instance, you might ask current customers what they like or don't like about your new product line. Or, you could ask them to give suggestions on how your organization could improve. They could also have the chance to elaborate on any pain points they're experiencing.

There are industry tools that allow you to code such open-ended data so users can understand it quantitatively. Yet, these don't come cheap. Word clouds offer a cost-effective, yet powerful, alternative.

With these, you can still quantify your text-based insights into measurable analytics. The only difference?
You won't create a chart or graph as you would with a set of numbers.

Instead, you'll create a word cloud generator to transform the most critical information into a word cloud.

Here's an example from USA Today using U.S. President Barack Obama's State of the Union Speech 2012:



As you can see, words like "American," "jobs," "energy" and "every" stand out since they were used more frequently in the original text.

Now, compare that to the 2014 State of the Union address:



V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY (50 POINTS)

Data Set:

- **Spam Email Data Set found here:**
  https://github.com/bendgame/nlpBeginnerProjects/blob/master/NLP_beginner_projects.ipynb

Steps:

1. Importing Dependencies and Data

Start by importing the dependencies and the data. The data is stored as a comma separated values (csv) file, so use pandas' read_csv() function to open it into a DataFrame.

```
import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud

#create dataframe from csv
df = pd.read_csv('emails.csv')

df.head()
```

	text	spam
0	Subject: naturally irresistible your corporate...	1
1	Subject: the stock trading gunslinger fanny i...	1
2	Subject: unbelievable new homes made easy im ...	1
3	Subject: 4 color printing special request add...	1
4	Subject: do not have money , get software cds ...	1

df.head()

2. Exploratory Analysis

Before anything, it is best to do a quick analysis of the data to eliminate duplicate rows and establish some baseline counts. Use pandas drop_duplicates to drop the duplicate rows.

```
print("spam count: " +str(len(df.loc[df.spam==1])))
print("not spam count: " +str(len(df.loc[df.spam==0])))
print(df.shape)
df['spam'] = df['spam'].astype(int)

df = df.drop_duplicates()
df = df.reset_index(inplace = False)[['text','spam']]


print(df.shape)
```

```
spam count: 1368
not spam count: 4360
(5728, 2)
(5695, 2)
```

Counts and shape before/after duplicate removal

3. Cleaning the Texts

Notice in the word cloud image, all the text is lower case. There are no punctuation marks or special characters. That's because the text has been cleaned to make it easier to analyze. Using regular expressions, it is easy to clean the text using a loop:

```
clean_desc = []
for w in range(len(df.text)):
    desc = df['text'][w].lower()

    #remove punctuation
    desc = re.sub('[^a-zA-Z]', ' ', desc)

    #remove tags
    desc=re.sub("&lt;/?.*?&gt;"," &lt;&gt; ",desc)

    #remove digits and special chars
    desc=re.sub("(\\d|\\W)+"," ",desc)

    clean_desc.append(desc)

#assign the cleaned descriptions to the data frame
df['text'] = clean_desc

df.head(3)
```

	text	spam
0	subject naturally irresistible your corporate ...	1
1	subject the stock trading gunslinger fanny is ...	1
2	subject unbelievable new homes made easy im wa...	1

4. Removing Stop Words

Stop words are the most common words like “the” and “of.” Removing them from the email text allows the more relevant frequent words to stand out. Removing stop words is a common technique! Some Python libraries like NLTK come pre-loaded with a list of stop words, but it is easy to create one from scratch.

```
stop_words = ['is','you','your','and', 'the', 'to', 'from', 'or',
'I', 'for', 'do', 'get', 'not', 'here', 'in', 'im', 'have', 'on',
're', 'new', 'subject']
```

5. Construct the Word Cloud

Conveniently, there is a Python library for creating word clouds. It can be installed using pip.

```
pip install wordcloud
```

When constructing the word cloud, it is possible to set several parameters like height and width, stop words, and max words. It is even possible to shape it instead of displaying the default rectangle.

```
wordcloud = WordCloud(width = 800, height = 800, background_color =
'black', stopwords = stop_words, max_words = 1000
                      , min_font_size =
20).generate(str(df1['text']))

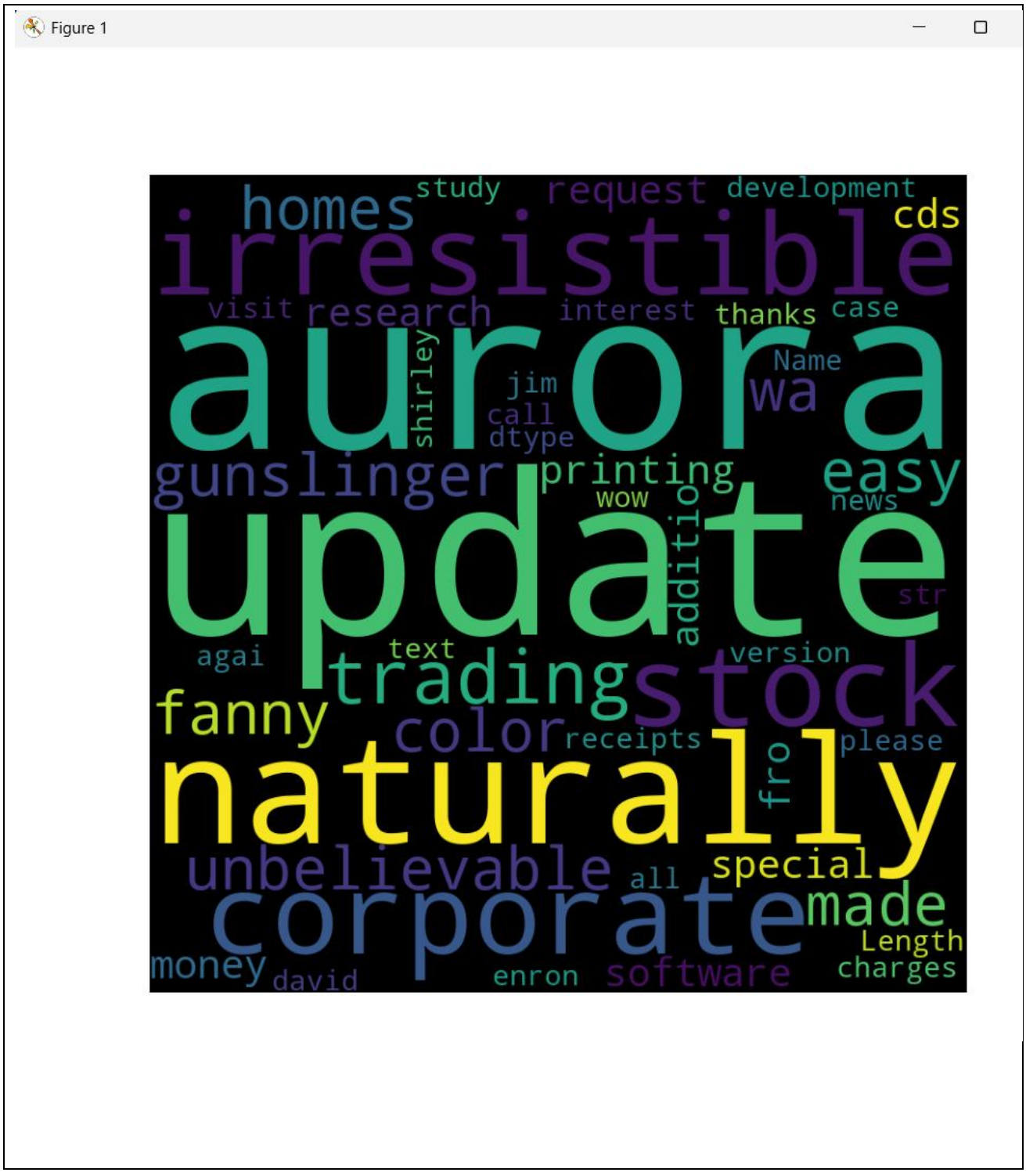

#plot the word cloud
fig = plt.figure(figsize = (8,8), facecolor = None)
plt.imshow(wordcloud)
plt.axis('off')
plt.show()
```

The word cloud can be saved and displayed using matplotlib and .show(). This is the result of all records, regardless of it being spam.



OUTPUT:

```
spam count: 1368
not spam count: 4360
(5728, 2)
(5695, 2)
```


Figure 1
homes
study
request
development
cds
irresistible
visit
research
interest
thanks
case
aurora
shirley
jim
wa
Name
call
dtype
gunslinger
printing
easy
wow
news
update
additio
str
agai
text
trading
stock
version
fanny
color
receipts
please
fro
naturally
unbelievable
all
special
corporate
made
Length
money
david
enron
software
charges

CODE:

```
import pandas as pd
import os
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud

script_folder = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(script_folder, 'emails.csv'))
df.head()

print("spam count: " +str(len(df.loc[df.spam==1])))
print("not spam count: " +str(len(df.loc[df.spam==0])))
print(df.shape)
df['spam'] = df['spam'].astype(int)

df = df.drop_duplicates()
print(df.shape)

df = df.reset_index(inplace = False)[['text','spam']]

clean_desc = []
for w in range(len(df.text)):
    desc = df['text'][w].lower()

    #remove punctuation
    desc = re.sub('[^a-zA-Z]', ' ', desc)

    #remove tags
    desc=re.sub("&lt;/?.*?&gt;"," &lt;&gt; ",desc)

    #remove digits and special chars
    desc=re.sub("(\\d|\\W)+"," ",desc)

    clean_desc.append(desc)

#assign the cleaned descriptions to the data frame
df['text'] = clean_desc
df = df.reset_index()
df.head(3)
```

```
df1 =df.loc[df.spam==0]
df2 =df.loc[df.spam==1]
stop_words = ['is','you','your','and', 'the', 'to', 'from', 'or', 'I', 'for', 'do', 'get', 'not',
'here', 'in', 'im', 'have', 'on', 're', 'new', 'subject']

# set the word cloud parameters
wordcloud = WordCloud(width = 800, height = 800, background_color = 'black',
stopwords = stop_words, max_words = 1000
                    , min_font_size = 20).generate(str(df['text']))

#plot the word cloud
fig = plt.figure(figsize = (8,8), facecolor = None)
plt.imshow(wordcloud)
plt.axis('off')
plt.show()
```

CODE SCREENSHOT:

```python
import pandas as pd
import os
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud

script_folder = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(script_folder, 'emails.csv'))
df.head()

print("spam count: " +str(len(df.loc[df.spam==1])))
print("not spam count: " +str(len(df.loc[df.spam==0])))
print(df.shape)
df['spam'] = df['spam'].astype(int)

df = df.drop_duplicates()
print(df.shape)

df = df.reset_index(inplace = False)[['text','spam']]

clean_desc = []
for w in range(len(df.text)):
    desc = df['text'][w].lower()

    #remove punctuation
    desc = re.sub('[^a-zA-Z]', ' ', desc)

    #remove tags
    desc=re.sub("&lt;/?.*?&gt;"," &lt;&gt; ",desc)

    #remove digits and special chars
    desc=re.sub("(\\d|\\W)+"," ",desc)

    clean_desc.append(desc)
```

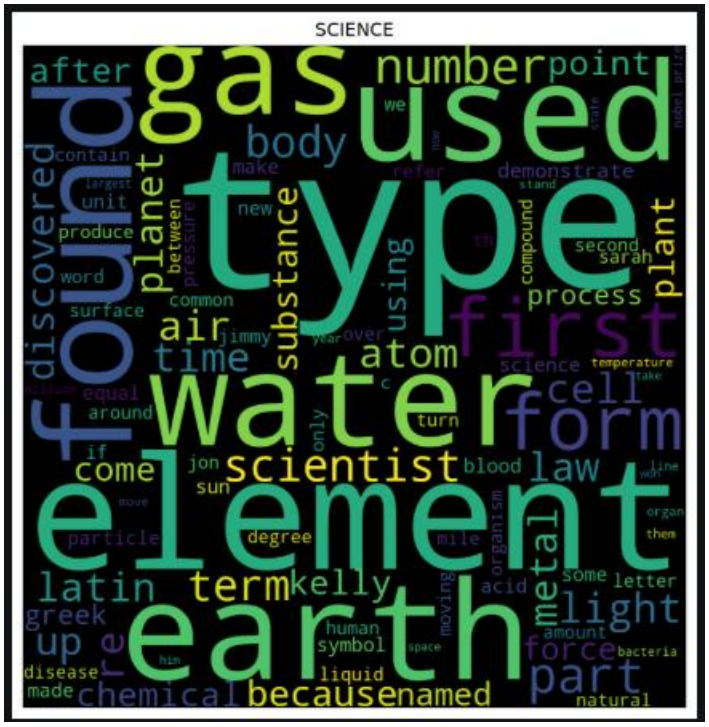
```
35
36  #assign the cleaned descriptions to the data frame
37  df['text'] = clean_desc
38  df = df.reset_index()
39  df.head(3)
40
41  df1 =df.loc[df.spam==0]
42  df2 =df.loc[df.spam==1]
43  stop_words = ['is','you','your','and', 'the', 'to', 'from', 'or', 'I', 'for', 'do', 'get', 'not', 'here', 'in', 'im', 'have', 'on', 're', 'new',
44
45  # set the word cloud parameters
46  wordcloud = WordCloud(width = 800, height = 800, background_color = 'black', stopwords = stop_words, max_words = 1000
47                        , min_font_size = 20).generate(str(df['text']))
48
49  #plot the word cloud
50  fig = plt.figure(figsize = (8,8), facecolor = None)
51  plt.imshow(wordcloud)
52  plt.axis('off')
53  plt.show()
```

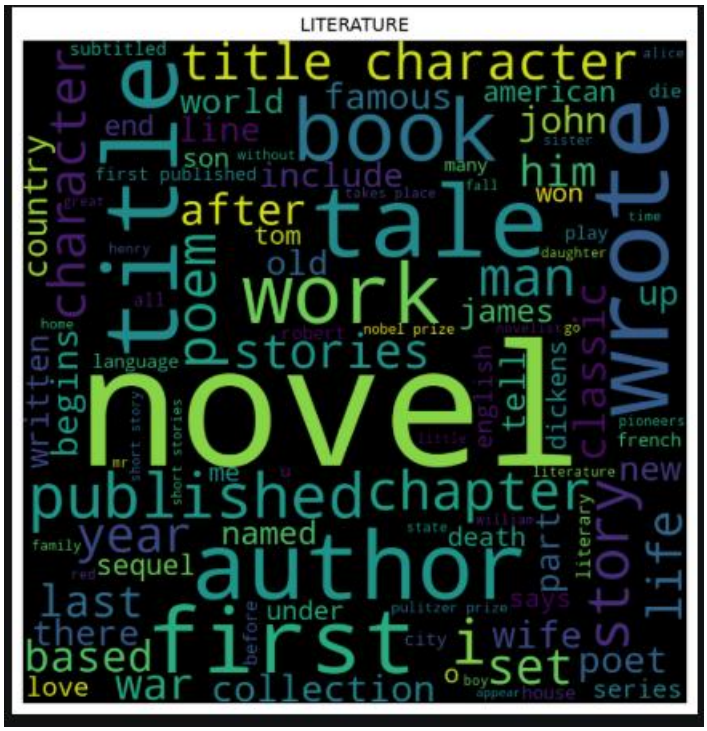
VII. CHALLENGE

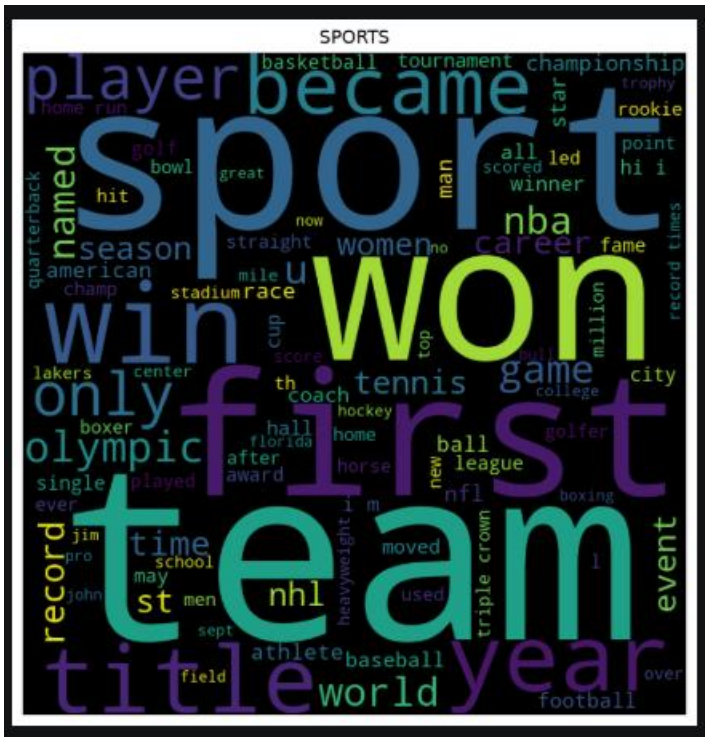
- Repeat the procedure you did in Part VI., this time using any of the following open data sets found in this URL: https://analyticsindiamag.com/10-nlp-open-source-datasets-to-start-your-first-nlp-project/
- Clearly specify the intention of the output that you will generate.
    - I chose this Jeopardy Questions dataset from the given link, and I chose this to compare the most frequent word between its three categories, which are science, literature, and sports.
- Compare and contrast sample outputs that you will generate.
- Provide a conclusion on whether you have achieved your goal or not.
    - Through WordCloud, yes, I gave the three categories a visual comparsion. Both categories have words that are in lined with their own category.
- Provide a recommendation on things you did wrong and suggestions to improve and achieve the expected output.
    - It probably would be the orientation of the words. A few words on each wordcloud are either in a horizontal or vertical orientation. Having a horizontal orientation on these words only will make the image comparison much better and understandable.
- You may use existing Word Cloud generator like Wordle, Tagxedo, Tag Crowd, WordItOut, Word Cloud Generator for Chrome, Word Cloud Python tools or Google Word Cloud tools and check your output against the output of the generators. Analyze and explain results.

OUTPUT:

```
total questions: 216930
after removing duplicates: 216132
saved cloud_science.png (515 questions)
saved cloud_literature.png (494 questions)
saved cloud_sports.png (341 questions)
```




LITERATURE
novel
title
wrote
book
tale
work
author
first
character
published
chapter
story
life
title character
world
famous
american
john
him
after
include
poem
man
stories
tell
classic
dickens
french
pioneers
new
year
named
sequel
death
last
under
there
based
war
collection
wife
set
poet
series
love
written
begins
country
end
line
son
tom
old
james
up
language
part
literary
says
i
o


SPORTS
sport
won
team
first
player
became
win
only
olympic
title
year
championship
basketball
tournament
named
season
nba
women
career
game
tennis
record
time
event
world
nhl
st
ball
league
nfl
coach
race
stadium
rookie
star
led
winner
hit
bowl
great
straight
now
fame
million
lakers
center
city
college
hockey
boxer
florida
home
after
award
horse
single
played
ever
jim
pro
school
may
men
moved
used
triple crown
heavyweight
athlete
baseball
football
field
over
sept
boxing
golfer
quarterback
american
champ
mile
cup
top
score
trophy
home run
golf
point
hi i
all
scored
no
record times

CODE:

```
import json
import re
import matplotlib.pyplot as plt
from wordcloud import WordCloud
import os

script_folder = os.path.dirname(os.path.abspath(__file__))
json_path = os.path.join(script_folder, "JEOPARDY_QUESTIONS1.json")

with open(json_path, "r") as json_file:
    all_questions = json.load(json_file)

print("total questions: " + str(len(all_questions)))

# removes duplicates
unique_questions = []
seen_lookup = {}
for item in all_questions:
    question_text = item["question"]
    if question_text not in seen_lookup:
        seen_lookup[question_text] = True
        unique_questions.append(item)

print("after removing duplicates: " + str(len(unique_questions)))

# cleans text
def clean_text(raw_text):
    # lowercase everything
    text = raw_text.lower()

    # remove html tags first (some clues have links like <a href=...>)
    text = re.sub("<[^>]+>", " ", text)

    # remove punctuation, digits, and special characters
    text = re.sub("[^a-z]", " ", text)

    # remove extra spaces
    text = re.sub("\\s+", " ", text)
    return text.strip()
```

```
for item in unique_questions:
    item["clean_question"] = clean_text(item["question"])

# stop words
stop_words = ["is", "you", "your", "and", "the", "to", "from", "or", "for", "do",
              "get", "not", "here", "in", "have", "on", "this", "that", "of", "a",
              "an", "as", "it", "its", "by", "with", "was", "were", "be", "are",
              "at", "his", "her", "he", "she", "their", "they", "one", "which",
              "who", "what", "when", "where", "name", "known", "called", "about",
              "than", "has", "had", "but", "into", "out", "these", "those", "can",
              "been", "also", "more", "most", "said", "say", "s", "t"]

# words that are only in this dataset and are not useful (found by looking at the
first clouds)
# "clue" and "crew" come from the "Clue Crew" video clues
jeopardy_stop_words = ["clue", "crew", "seen", "like", "called", "jeopardy"]

# builds the wordcloud
def make_word_cloud(question_list, title, file_name, extra_stop_words):
    # join all the cleaned questions into one big string
    # (we put them in a list first and join once, because adding to a string
    # one by one is very slow when there are almost 100,000 questions)
    all_clean_questions = []
    for item in question_list:
        all_clean_questions.append(item["clean_question"])
    big_text = " ".join(all_clean_questions)

    all_stop_words = set(stop_words + extra_stop_words)

    word_cloud = WordCloud(width=800, height=800, background_color="black",
                           stopwords=all_stop_words, max_words=100,
                           min_font_size=10).generate(big_text)

    plt.figure(figsize=(8, 8), facecolor=None)
    plt.imshow(word_cloud)
    plt.title(title)
    plt.axis("off")
    plt.savefig(file_name, dpi=120, bbox_inches="tight")
    plt.close()
```

```
    print("saved " + file_name + " (" + str(len(question_list)) + " questions)")

# comparison of categories: science, literature, and sports
science_questions = []
literature_questions = []
sports_questions = []
for item in unique_questions:
    if item["category"] == "SCIENCE":
        science_questions.append(item)
    elif item["category"] == "LITERATURE":
        literature_questions.append(item)
    elif item["category"] == "SPORTS":
        sports_questions.append(item)

make_word_cloud(science_questions, "SCIENCE", "cloud_science.png",
jeopardy_stop_words)
make_word_cloud(literature_questions, "LITERATURE", "cloud_literature.png",
jeopardy_stop_words)
make_word_cloud(sports_questions, "SPORTS", "cloud_sports.png",
jeopardy_stop_words)
```

CODE SCREENSHOT:

```python
import json
import re
import matplotlib.pyplot as plt
from wordcloud import WordCloud
import os

script_folder = os.path.dirname(os.path.abspath(__file__))
json_path = os.path.join(script_folder, "JEOPARDY_QUESTIONS1.json")

with open(json_path, "r") as json_file:
    all_questions = json.load(json_file)

print("total questions: " + str(len(all_questions)))

# removes duplicates
unique_questions = []
seen_lookup = {}
for item in all_questions:
    question_text = item["question"]
    if question_text not in seen_lookup:
        seen_lookup[question_text] = True
        unique_questions.append(item)

print("after removing duplicates: " + str(len(unique_questions)))

# cleans text
def clean_text(raw_text):
    # lowercase everything
    text = raw_text.lower()

    # remove html tags first (some clues have links like <a href=...>)
    text = re.sub("<[^>]+>", " ", text)

    # remove punctuation, digits, and special characters
    text = re.sub("[^a-z]", " ", text)
```

```
    # remove extra spaces
    text = re.sub("\\s+", " ", text)
    return text.strip()


for item in unique_questions:
    item["clean_question"] = clean_text(item["question"])

# stop words
stop_words = ["is", "you", "your", "and", "the", "to", "from", "or", "for", "do",
              "get", "not", "here", "in", "have", "on", "this", "that", "of", "a",
              "an", "as", "it", "its", "by", "with", "was", "were", "be", "are",
              "at", "his", "her", "he", "she", "their", "they", "one", "which",
              "who", "what", "when", "where", "name", "known", "called", "about",
              "than", "has", "had", "but", "into", "out", "these", "those", "can",
              "been", "also", "more", "most", "said", "say", "s", "t"]

# words that are only in this dataset and are not useful (found by looking at the first clo
# "clue" and "crew" come from the "Clue Crew" video clues
jeopardy_stop_words = ["clue", "crew", "seen", "like", "called", "jeopardy"]

# builds the wordcloud
def make_word_cloud(question_list, title, file_name, extra_stop_words):
    # join all the cleaned questions into one big string
    # (we put them in a list first and join once, because adding to a string
    # one by one is very slow when there are almost 100,000 questions)
    all_clean_questions = []
    for item in question_list:
        all_clean_questions.append(item["clean_question"])
    big_text = " ".join(all_clean_questions)

    all_stop_words = set(stop_words + extra_stop_words)

    word_cloud = WordCloud(width=800, height=800, background_color="black",
```

```
                                stopwords=all_stop_words, max_words=100,
                                min_font_size=10).generate(big_text)

    plt.figure(figsize=(8, 8), facecolor=None)
    plt.imshow(word_cloud)
    plt.title(title)
    plt.axis("off")
    plt.savefig(file_name, dpi=120, bbox_inches="tight")
    plt.close()
    print("saved " + file_name + " (" + str(len(question_list)) + " questions)")

# comparison of categories: science, literature, and sports
science_questions = []
literature_questions = []
sports_questions = []
for item in unique_questions:
    if item["category"] == "SCIENCE":
        science_questions.append(item)
    elif item["category"] == "LITERATURE":
        literature_questions.append(item)
    elif item["category"] == "SPORTS":
        sports_questions.append(item)

make_word_cloud(science_questions, "SCIENCE", "cloud_science.png", jeopardy_stop_words)
make_word_cloud(literature_questions, "LITERATURE", "cloud_literature.png", jeopardy_stop_wo
make_word_cloud(sports_questions, "SPORTS", "cloud_sports.png", jeopardy_stop_words)
```

VIII. REFERENCES

- Vahid Mirjalili(2017). Python Machine Learning: Machine Learning and Deep Learning with Python, scikitlearn, and TensorFlow. India: Packt Publishing
- Numerical Algorithms: Methods for Computer Vision, Machine Learning, and Graphics

Online Sources:

- https://boostlabs.com/blog/what-are-word-clouds-value-simple-visualizations/
- https://github.com/bendgame/nlpBeginnerProjects/blob/master/NLP_beginner_projects.ipynb
- https://towardsdatascience.com/3-super-simple-projects-to-learn-natural-language-processing-usingpython-8ef74c757cd9

10